Voice Automatic Gain Control (AGC) Software Users Guide

Prepared by Adaptive Digital Technologies, Inc.
January 21, 2016

Issue 2

Copyright © 2001 - 2019, Adaptive Digital Technologies Inc.

Revision History

Issue	Revision History	Date	Initials
1	Clean start – V3.00	11/16/2011	JDC
2	Get API Version, Get Library version return unsigned int	1/21/2016	JDC

Table of Contents

1. INTRODUCTION 1

2. SOFTWARE INTERFACE 1

2.1 MEMORY ALLOCATION 1
2.2 INITIALIZATION 2
2.3 GET LIBRARY VERSION 4
2.4 GET API VERSION 4
2.5 APIS FOR RUNNING THE AGC 4
2.5.1 Combined Interface 5
2.5.2 Split Interface 5
2.6 SAMPLE HOST SOFTWARE 6

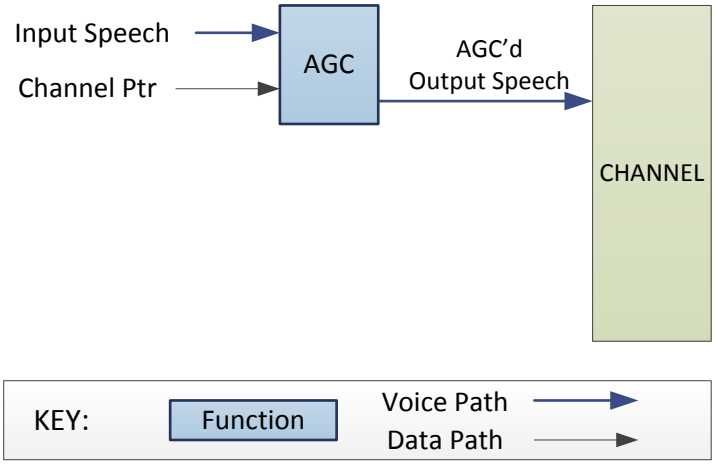
1. Introduction

The Adaptive Digital Technologies (ADT) Automatic Gain Control (AGC) algorithm is used to automatically adjust the speech level of an audio signal so that the level falls within a user-defined output level range. In voice networks, audio signal paths may have different gains or losses. The AGC algorithm can be used to adjust the level of a speech signal to a more desirable level.

This document describes the AGC software in order to enable a software engineer to integrate the software into a host DSP application.

2. Software Interface

The AGC voice channel is show in the block diagram below.



2.1 Memory Allocation

The host software must allocate persistent memory, known as a channel or instance structure, for each instance of a voice channel. The instance data structure can be found in "C" include file agc_user.h

Instantiation of Channel Structure

A separate instance data structure must be allocated for each active voice channel.

Instantiation:

```
#include "agc_user.h"
#define NUM_CHANNELS 10
```

.
.
AGCInstance_t TheChannel[NUM_CHANNELS]; // allocate multiple channels

2.2 Initialization

An initialization function is provided to initialize the AGC channel structures. The function prototype is:

Prototype:

```
ADT_UInt8 AGC_ADT_agcinit (AGCInstance_t *AgcChan_p,
                           AGCParamV3_00_t *pParams);
```

```
typedef struct
{
   ADT_UInt32 APIVersion;
   ADT_UInt32 instanceSizeI8;   // InstanceSize in bytes
   ADT_Int16  targetPowerLowdBm;  // INPUT range: 0 ... -30
   ADT_Int16  targetPowerHighdBm; // INPUT range: 0 ... -30
   ADT_Int16  maxLossdB;          // INPUT range: 0 ... -23
   ADT_Int16  maxGaindB;          // INPUT range: 0 ... +23
   ADT_Int16  lowSigThreshdBm;    // INPUT threshold in dBm of low
                                    // energy signal
   ADT_Int16  stickyControl;      // INPUT Flag, 0/1/2, to remember gain
                                    // when signal below above
   ADT_Int16  stickyInitialGaindB;// INPUT range: -23 ... +23
   ADT_Int32  samplingRateHz;     // INPUT sampling rate of the
                                    // input/output data
} AGCParamV3_00_t;
```

Inputs:

AgcChan_p - Pointer to the channel Instance to be initialized.

pParams – Pointer to initialization parameter structure, which is defined in the table below

ElementName	Description	Recommended / Typical Value
APIVersion	Version of the AGC API. This enables the initialization function to ensure that the user is using a header file that is appropriate for the AGC library that is being used	AGC_API_VER
instanceSizeI8	Size of the Instance Structure (AGCInstance_t) in bytes that is being used. This allows the initialization function to check to make sure that enough memory is allocated to the instance structure.	Sizeof(AGCInstance_t)
targetPowerLowdBm targetPowerHighdBm	When the input signal level exceeds the noise threshold (lowSigThreshdBm), the AGC algorithm tries to drive the output level so that it remains in the range between these two values, specified in dBm. Range is -30 .. 0 dBm. targetPowerLowdBm must be greater than targetPowerHighdBm.	-20, -10, respectively
maxLossdB	The maximum loss applied by the AGC, specified in dB. Range is 0..23	6
maxGaindB	The maximum gain applied by the AGC, specified in dB. Range is 0..23	15
lowSigThreshdBm	The input signal level, below which, the AGC will not apply gain. This is done to avoid applying gain to noise when speech isn't present. Range is -65 .. -20 dBm	-35 dBm
stickyControl	0: The AGC will not remember the gain of a previous speech burst once the input level drops below lowSigThreshdBm 1: The AGC will remember the gain of a previous speech burst, and when speech returns after a speech-free period, the remembered gain will be applied immediately without need for ramp-up 2: The AGC will remember the gain of a previous speech burst and apply that gain during a non-speech burst as well as to the subsequent speech burst	1
stickyInitialGaindB	The initial gain value for use when stickyControl is either 1 or 2. 0..23	0
samplingRateHz	The sampling rate of the signal, specified in Hz. The allowable sampling rates are defined in agc_user.h	8000

Outputs: None

Returns Result Code (self-explanatory)

```
#define AGC_INIT_OK 0
#define AGC_INIT_ERR_WRONG_API_VERSION 1
#define AGC_INIT_INSTANCE_SIZE_TOO_SMALL 2
#define AGC_INIT_UNSUPPORTED_SAMPLING_RATE 3
```

2.3 Get Library Version

Prototype:

ADT_UInt16 AGC_ADT_getLibraryVersion( )

Inputs: None

Outputs: None

Returns: The library build version

2.4 Get API Version

Prototype:

ADT_UInt16 AGC_ADT_getAPIVersion( )

Inputs: None

Outputs: None

Returns: The API build version

2.5 APIs for Running the AGC

There are two ways to run the AGC (once it has been initialized). The most common way is with a single function call to AGC_ADT_agcRun. But under some circumstances the user may require finder control of the signal flow. In this case, we split the "run" function into two functions: AGC_ADT_agcComputeGain and AGC_ADT_agcApplyGain. The names are self-explanatory. The split API is useful when integrating the AGC with echo cancellers. In this case, it may be desirable to compute the gain based upon the residual signal before any nonlinear processing (NLP) or is applied or any comfort noise generation (CNG). Yet the gain can be applied (perhaps conditionally) after the NLP/CNG have operated on the signal.

Setup (at start of each voice call):

The AGC software must be initialized using `AGC_ADT_agcInit()` before calling these AGC APIs.

Frame Rate Processing:

2.5.1 Combined Interface

The AGC algorithm is invoked by calling the `AGC_ADT_agcRun`. The function prototype is:

Prototype:

```
void AGC_ADT_agcRun (AGCInstance_t *AgcChan_p,
                     ADT_Int16 *InputSamples_p,
                     ADT_Int16 NAGCSamples,
                     ADT_Int16 ForceLowSignalState,
                     ADT_Int16 BlockShift,
                     ADT_Int16 *OutputSamples_p,
                     ADT_Int32 *Gains_p);
```

Input:

AgcChan_P – pointer to the AGC channel instance
***InputSamples_p** – The input speech data consisting of uniformly quantized 16-bit samples
NAGCSamples – The number of input and output samples.
ForceLowSignalState – This flag, if set to 1, will override the lowSigThreshDBM threshold check and force the AGC into the low signal state.
BlockShift – block floating-point exponent. A positive number indicates that the input samples have been normalized by scaling them down. A negative number indicates that the input samples have been normalized by scaling them up. It should be noted that for most applications, a value of zero should be used.

Output:

***OutputSamples_p –** The AGC'd speech data consisting of uniformly quantized 16-bit samples. This pointer may be the same as the InputSamples_p[ ] for in-place computations.
***Gains_p –** This pointer holds the starting address of the array in which individual gains for the samples are stored. Normally this pointer is set to zero, and no gains are returned.

2.5.2 Split Interface

Alternately, the AGC algorithm can be invoked by calling the following two functions:

Prototype:

```
void AGC_ADT_agcComputeGain (AGC_ChanInst_t *AgcChan_p,
                     ADT_Int16 *InputSamples_p,
                     ADT_Int16 NAGCSamples,
                     ADT_Int16 BlockShift);
```

Input:

AgcChan_P – pointer to the AGC channel instance
***InputSamples_p** – The input speech data consisting of uniformly quantized 16-bit samples
NAGCSamples – The number of input and output samples.

BlockShift – block floating-point exponent. A positive number indicates that the input samples have been normalized by scaling them down. A negative number indicates that the input samples have been normalized by scaling them up. It should be noted that for most applications, a value of zero should be used.

Output: None

Prototype:

```
void AGC_ADT_agcApplyGain(AGCInstance_t*AgcChan_p,
                    ADT_Int16 *InputSamples_p,
                    ADT_Int16 NAGCSamples,
                    ADT_Int16 ForceLowSignalState,
                    ADT_Int16 *OutputSamples_p,
                    ADT_Int32 *Gains_p);
```

Input:

AgcChan_P – pointer to the AGC channel instance
***InputSamples_p** – The input speech data consisting of uniformly quantized 16-bit samples
NAGCSamples – The number of input and output samples.
ForceLowSignalState – This flag, when set to 1, will override the AGC's low signal detection. When the AGC is used in conjunction with an echo canceller, this flag can be set during double-talk and/or during NLP/CNG operation.

Output:

***OutputSamples_p –** The AGC'd speech data consisting of uniformly quantized 16-bit samples. This pointer may be the same as the InputSamples_p[ ] for in-place computations.
***Gains_p –** This pointer holds the starting address of the array in which individual gains for the samples are stored.

2.6 Sample Host Software

```
#include "agc_user.h"

AGCInstance_t AGCInstance;
AGCParamV3_00_t Params;

#define TARGET_POWER_LOW -25
#define TARGET_POWER_HIGH -10
#define MAX_LOSS_LIMIT_IN 6
#define MAX_GAIN_LIMIT_IN 20
#define FRAME_SIZE 80
#define LOW_SIG_THRESH_DBM -30
#define STICKY_CONTROL 2
#define STICKY_INITIAL_GAIN 0
#define SAMPLING_RATE 8000

ADT_Int16 InFrame[FRAME_SIZE],OutFrame[FRAME_SIZE];
ADT_Int32 Gains[FRAME_SIZE];
ADT_UInt8 ReturnValue;
```

```
Params.APIVersion = AGC_API_VER;
Params.instanceSizeI8 = sizeof(AGCInstance);
Params.targetPowerLowdBm = TARGET_POWER_LOW;
Params.targetPowerHighdBm = TARGET_POWER_HIGH;
Params.maxLossdB = MAX_LOSS_LIMIT_IN;
Params.maxGaindB = MAX_GAIN_LIMIT_IN;
Params.lowSigThreshdBm = LOW_SIG_THRESH_DBM;
Params.stickyControl = STICKY_CONTROL;
Params.stickyInitialGaindB = STICKY_INITIAL_GAIN;
Params.samplingRateHz = SAMPLING_RATE;

ReturnValue = AGC_ADT_agcInit(&AGCInstance, &Params);

AGC_ADT_agcComputeGain(&AGCInstance, InFrame, FRAME_SIZE, 0);
AGC_ADT_agcApplyGain(&AGCInstance, InFrame, FRAME_SIZE, 0, OutFrame, Gains);
```